algolab: an empirical analysis of sorting, bin packing, and network structure.

Will Clark

August 5, 2026

Contents

1 Sorting Algorithms

This report analyzes the empirical running times of insertion sort, several variations of shell sort, Tim-sort, and skip-list sort with their theoretical complexity. Each algorithm is measured across three input distributions, namely: uniformly distributed, nearly-sorted, and two-alternating. Finally, I declare a best algorithm.

1.1 Introduction

Sorting algorithms are the *foundation* of computer science. As per Donald Knuth, "virtually every important aspect of programming arises somewhere in the context of sorting or searching!" And not only that, the speed at which we can sort data is often the limiting factor in modern advancements. When discussing the "speed" of an algorithm, a natural crossroads appears: *theoretical* speed, or *practical* speed. This report seeks to derive insights from theoretical explanations, and apply them to empirical results. The canonical choice for analysis is *time vs. input size*, but I'd like to append *comparison count vs. input size* to our analysis. Both cases are plotted on log-log scales to make spotting power-law ($T(n) = cn^k$) running times easier. The below algorithms – and theoretical (worst-case) running times – are listed for reference later.

1.2 Setup

1.2.1 Input Distributions

A `Permutation` class was created to handle all input distributions. Each `Permutation` instance stores a `random.Random(SEED)` into `self._rng`. `Permutation` has member functions `uniform`, `two_alt`, and `almost_sorted`, yielding a uniformly distributed sequence, almost sorted sequence (bounded by $\log_2(|S|)$ swaps), and a two-alternating sequence $(1, 3, 5, \ldots, n-1, 2, 4, 6, \ldots, n)$, respectively. Since Python implements the Mersenne Twister for its random number generation, we don't have to worry about any LCG-adjacent issues.

1.2.2 Data Collection

I used `Pydantic` to store comparisons and time per iteration for every algorithm into `JSON`. Pydantic gave me more flexibility than the suggested `csv` file in the notes. I tested each of the algorithms for 30 trials, on input sizes $16, 32, 64, 128..., 16384$ as to align them with a $\log_2$ axis when graphing.

I) **Insertion Sort**: $\mathcal{O}(N^2)$

II) **Shell Sort** with gap sequence h_k:

- i) **Shell's original** $h_k = \lfloor \frac{N}{2^k} \rfloor$, $k = 1, 2, \ldots, \lfloor \log_2 N \rfloor$: $\mathcal{O}(N^2)$, for $N = 2^p$, $p \in \mathbb{Z}^+$.
- ii) **Frank and Lazarus** $h_k = 2\lfloor \frac{N}{2^{k+1}} \rfloor + 1$, yielding $1, 3, \ldots, 2\lfloor \frac{N}{4} \rfloor + 1$: $\mathcal{O}(N^{3/2})$.
- iii) **A083318** $h_k = 2^k + 1$, with 1, yielding $1, 3, 5, 9, 17, \ldots$: $\mathcal{O}(N^{3/2})$.
- iv) **A003586** $2^p 3^q$ (3-smooth), yielding $1, 2, 3, 4, \ldots$: $\mathcal{O}(N \log^2 N)$.
- v) **A003462** $h_k = \frac{3^k - 1}{2} \leq \lceil \frac{N}{3} \rceil$, yielding $1, 4, 13, 40 \ldots$: $\mathcal{O}(N^{3/2})$.

III) **Tim Sort**: $\mathcal{O}(N \log N)$ worst case, $\mathcal{O}(N)$ best case (adaptive, stable).

IV) **Skip List Sort**: $\mathcal{O}(N \log N)$ expected, $\mathcal{O}(N^2)$ worst case.

Figure 1: Asymptotic complexity of Sorting Algorithms

1.2.3 Graphing

Using the `JSON` data, I generated log-log plots for *Comparisons vs. Input Size*, and *Time(s) vs. Input Size* using `Matplotlib.pyplot`. Each plot has three best fit lines, one for each input distribution. To avoid cluttering, I am only including the *Time(s) vs. Input Size* plots, but will reference the other graphs if needed.

1.3 Insertion Sort

Take c to be the coefficient in front of $\log N$. Insertion sort performed the worst on the uniform distribution ($c = 2.0195$), closely followed by the two-alternating sequence ($c = 1.9956$), then further away by the near-sorted list ($c = 1.2174$). This directly follows from the theory: our empirical running time for uniform and two-alternating is close to $\mathcal{O}(n + k) \subseteq \mathcal{O}(n^2)$, but when the list is almost sorted we have close to linear, $\mathcal{O}(n)$ run-time, since we don't have to "make room" when fixing an inversion, k. When the input is two-alternating, insertion sort scans halfway up the list (until it reaches

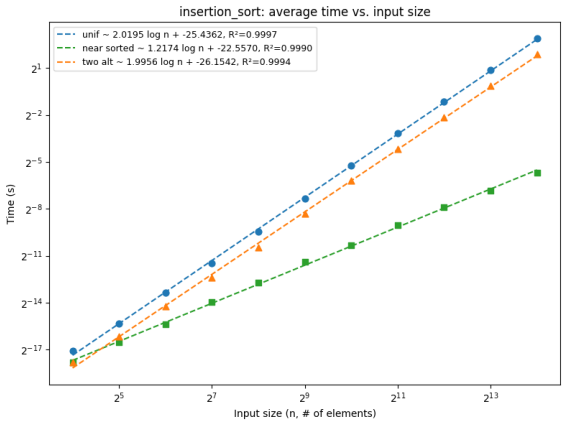


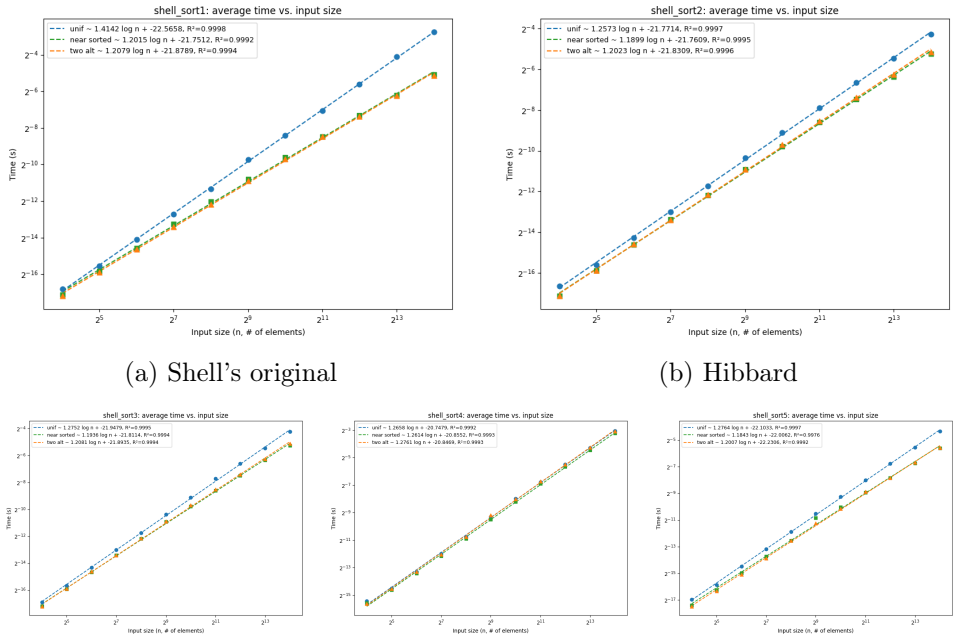
Figure 2: Insertion Sort

S_{n-1}), then has to do $\mathcal{O}(n^2)$ work moving over the odd numbers to insert the even ones. As such, this suggests that insertion sort *is* a viable sort on nearly-sorted sequences, less viable on two-alternating, and should be feared for uniformly distributed inputs.

1.4 Shellsort Variants

In all of the shell-sort variations, the **nearly-sorted input took the least time**, boasting values of $c = 1.2015, 1.1899, 1.1956, 1.2614, 1.1843$ for each variant, respectively. **A003462** was the **best among all variants for nearly sorted and two-alternating** inputs. **Hibbard** was the **best for uniform** inputs. Here's where things get strange, Shell's original formulation has empirical time complexity for uniform, nearly sorted, and two alternating of $\mathcal{O}_{emp}(n^{1.4102}), \mathcal{O}_{emp}(n^{1.2015}), \mathcal{O}_{emp}(n^{1.2079})$, which is much less than its theoretical $\mathcal{O}(n^2)$ bound. This allows me to conclude that on the average case, Shell-sort is quite an efficient sort. Another observation is that A003586 is almost *input-agnostic*, in the sense that each input produces the same, consistent $\mathcal{O}(n^{\approx 1.26})$ time bound. Since the 3-smooth gaps $\{2^a \cdot 3^b\}$ are dense across all scales, no input permutation can consistently evade early reduction passes, yielding stable $\mathcal{O}(n^{\approx 1.26})$ regardless of input structure.

1.5 Timsort

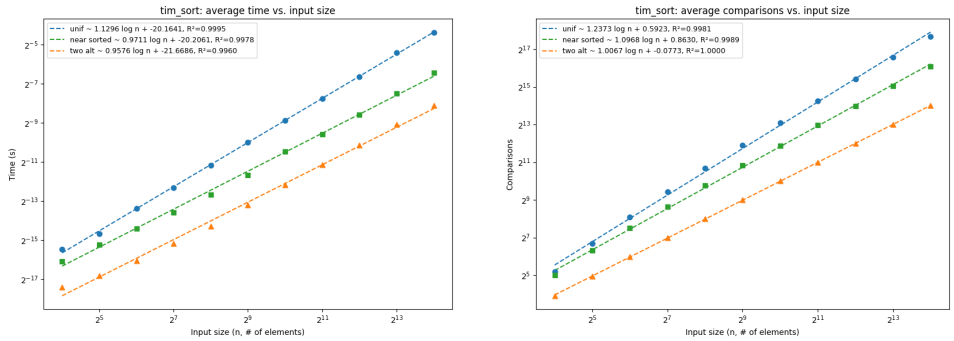


(a) Shell's original (b) Hibbard

(c) A083318 (d) A003586 (e) A003462

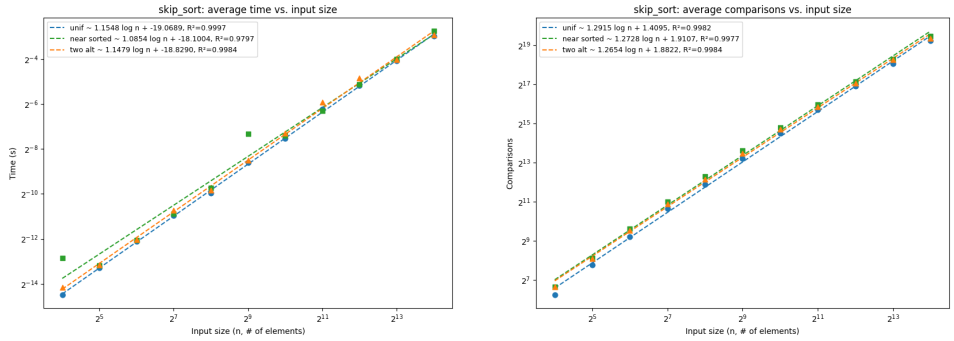
Figure 3: Log-log runtime plots for the Shellsort gap sequences.

The literature tells us that Tim Sort runs in $\mathcal{O}(n + n\mathcal{H}) \subseteq \mathcal{O}(n \log n)$ where $\mathcal{H}$ is the Shannon entropy, but upon a naive glance at the runtime of Tim Sort on the *Time vs. Input Size* plot, we get empirical time complexity of $\mathcal{O}_{emp}(n^{1.1296})$, $\mathcal{O}_{emp}(n^{0.9711})$, and $\mathcal{O}_{emp}(n^{0.9576})$. It's impossible for a sort to be *this* good, so looking at the comparisons plot, we get: $\mathcal{O}_{emp}(n^{1.2373})$, $\mathcal{O}_{emp}(n^{1.0968})$, and $\mathcal{O}_{emp}(n^{1.0067})$. Tim Sort almost runs in $\mathcal{O}(n)$ in the nearly sorted and two-alternating cases. I conjecture that this is because we only have 2 *monotonic non-decreasing* runs in the two-alternating case (the odd half $1, 3, \ldots, n-1$ followed by the even half $2, 4, \ldots, n$), so we are effectively only doing **ONE** merge. It's not quite as extreme in the near-sorted case, but the same logic applies: nearly sorted $\rightarrow$ less runs. On log-log plots, it's quite hard to distinguish between linear and logarithmic functions, but since the exponent, $c \in [1, 1.24]$, it's reasonable to assume an $\mathcal{O}(n \log n)$ average case runtime, but if the input is advantageous, **Tim sort is incredible.**



(a) Runtime (b) Comparisons

Figure 4: Tim Sort Plots



(a) Runtime (b) Comparisons

Figure 5: Skip List Sort Plots

1.6 Skip List Sort

Skip sort performs incredibly well on all input distributions, validating its "adaptive-sort" classification. The theory tells us that Skip sort has a complexity of $\mathcal{O}(n \log n)$ *with high probability*. It's not deterministic since for each input element we repeat *Bernoulli*$(\frac{1}{2})$ until we get 0, which could be infinite if you're (extremely) unlucky. As a general trend, note the $\approx .15$ increase between plots (a) and (b). Since skip sort performs `insert(x)` (and thus `search(x)`) for each element, there is some comparison overhead in actually constructing the skip list that is not reflected in runtime.

Let us now compare skip sort to the rest of the algorithms on each input distribution. For **uniform**, skip sort outperforms every sort except for Tim

sort, and by similar reasoning as *Section 5*, we can assume an $\mathcal{O}(n \log n)$ run time, which is strictly less than Tim Sort. For **Almost Sorted**, we only lose to Tim sort in for runtime complexity, but for comparisons we fall middle of the pack. For **Two-Alternating**, we fall right behind Tim Sort but beat everyone else.

1.7 Input Distribution Sensitivity

To address sensitivity, I split the algorithms into two camps: *input-sensitive* (whose empirical exponent c varies meaningfully across distributions) and *input-agnostic* (whose c is roughly constant). I use the spread $\Delta c = c_{\max} - c_{\min}$ across the three distributions as a rough proxy.

Input-sensitive algorithms. Insertion sort is the most dramatic case, with $\Delta c \approx 0.80$ ($c_{\text{unif}} = 2.02$, $c_{\text{near}} = 1.22$, $c_{\text{two-alt}} = 2.00$). This is expected: insertion sort's running time is governed by the inversion count k, which collapses to $\mathcal{O}(n)$ on nearly-sorted input but blows up to $\Theta(n^2)$ on uniform and two-alternating sequences. Tim sort is also clearly sensitive ($\Delta c \approx 0.23$ in comparisons), since its merge structure exploits existing runs – the two-alternating case degenerates to essentially a single merge, and the nearly-sorted case has few long runs to combine. Skip-list sort shows mild sensitivity ($\Delta c \approx 0.07$ in time), which is consistent with its $\mathcal{O}(n \log n)$ expected-time guarantee being only weakly dependent on input order.

Input-agnostic algorithms. Among the Shellsort variants, A003586 (Figure 3d) is the standout, with all three exponents collapsing to $c \approx 1.26$ ($\Delta c < 0.01$). As argued in Section 4, the density of 3-smooth integers $\{2^a 3^b\}$ across all scales prevents any input permutation from systematically evading early reduction passes. The other Shellsort variants (Shell's original, Hibbard, A083318, A003462) sit in between: their nearly-sorted exponents are noticeably lower (≈ 1.19) than uniform (≈ 1.40 for Shell's original, ≈ 1.26 for the others), but the gap is much smaller than insertion sort's.

In summary, sensitivity tracks the algorithm's ability to exploit existing order. Insertion sort and Tim sort are the most adaptive (and thus most sensitive); A003586 Shellsort is the least adaptive (and thus most agnostic); skip-list sort sits in the middle, gaining little from structure but losing little to the lack of it.

1.8 Best Algorithm

Tim Sort is the best algorithm we studied, since it adapts to the input distribution's structure and runs incredibly fast. From a philosophical standpoint,

there **cannot** be any "best" sorting algorithm, since the context in which you'd use a specific sort is non-static. Also, upon a simple Google search, Powersort replaced Tim Sort in `Python 3.11`, so even in its general domain, Tim-Sort is not the best.

2 Bin Packing

This report analyzes the waste generated by five bin packing algorithms: *Next Fit, First Fit, Best Fit, First Fit Decreasing* and *Best Fit Decreasing.*

2.1 Introduction

The *Bin Packing* problem is an optimization problem wherein items of different sizes must be *packed* into a finite number of *bins* with a certain capacity. Formally, take $I = \{i_1, \dots i_n\}$ to be a set of n items, where for each $i_k \in I$, a function s

$$\begin{aligned} s\colon I &\to (0, 1] \\ i_k &\mapsto s(i_k) \end{aligned}$$

denotes the size of the item. We want to know, for a set of bins $B = \{b_1, \dots, b_m\}$ with initial capacities $c(b_i) = 1$ for all $i \in \{1, \dots, m\}$, the miniumum number of bins required to pack all $|I|$ items.

Despite its simple nature, the problem is computationally **NP-Hard**, meaning that its incredibly unlikely a polynomial time algorithm exists to *optimally* solve it. There are, however, many approximation algorithms we can use to *approximate* bin packing, of which five will be discussed in later sections.

2.1.1 Waste

The goal of each of the experiments is to provide empirical evidence for estimating the *Waste* of a bin-packing algorithm. Let $\mathbb{I}$ denote the set of all sequences of valid items and $\mathbb{A}$ the set of all algorithms.

$$W\colon (\mathbb{A}, \mathbb{I}) \to \mathbb{R}^+ \cup \{0\}$$

be the waste of an algorithm, A, where for a set of items I:

$$W(A, I) = |\text{Bins Used by } A| - \sum_{i \in I} s(i)$$

We define waste as to compare approximation algorithms, instead of those that find the optimal solution. This is more valuable to reasoning which of the superceding algorithms is best, since computing an approximation ratio of an algorithm

$$\frac{ALG(I)}{OPT(I)}$$

is contingent on knowing the optimal solution, which takes exponential time to compute.

2.1.2 Data Structures

Traditional *Best Fit* and Next Fit variations run in $\mathcal{O}(n^2)$ time. We utilize a *Zip Zip Tree* to optimize both algorithms. Formally, a *Zip Zip Tree* is a binary search tree where every node has a numeric *rank*, and the tree is max-heap ordered with respect to *rank*.

Zip Trees, in their very nature, are constructed in such a way as to emulate a perfect binary tree. Theory tells us that perfect binary tree's have height $h = \log_2(n)$, for n vertices. *Zip Trees* invert this fact, and let *rank* be a geometric random variable with success probability $p = \frac{1}{2}$. To break collisions, we let *rank* be a tuple (r_1, r_2) where

$$\begin{aligned} r_1 &\sim Geom(\frac{1}{2}), && \mathcal{O}(\log\log n) \\ r_2 &\sim Unif[0, \log^c n], && \mathcal{O}(c\log\log n) \end{aligned}$$

and compare nodes lexigraphically. *Zip Trees* support `insert`, `find`, and `delete` with help from the `zip` operation. *Zipping* and *Unzipping* preserve the order and max-heap properties. *Zip Trees* are very efficient andeasyto implement, and, arguably more importantly, are isomorphic to skip-lists of which were implemented in *Project 1: Sorting*.

2.2 Next Fit

Next Fit is the most original hueristic aprroximation algorithm. It is simply:

- Try to place item i into bin b
- If it fits, put it there.
- If not, start a new bin.

The approximation ratio for Next Fit is $SOL = k < 2M$, where M is the optimal number of bins. The *Waste* grows linearly

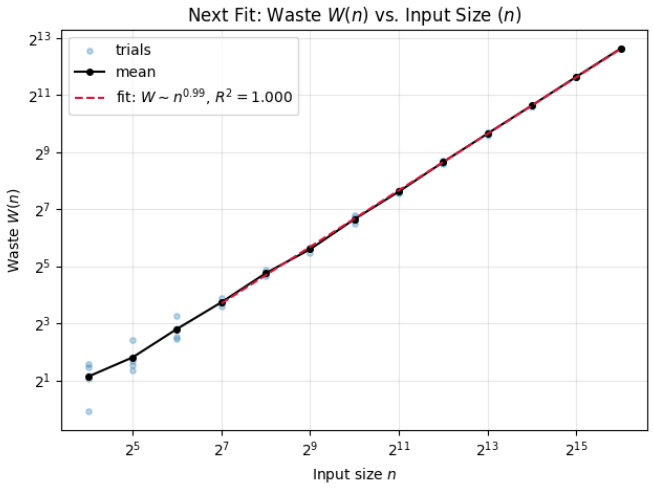


Figure 6: Next Fit Waste

as noted by the figure above. This is in direct agreement with the approximation ratio since there is N waste for an input size of N, meaning we are bounded by N extra buckets in the worst case (all waste requires a new bucket). This is exactly the bound above,

$$W(NF, A) \propto N, \qquad SOL = k < 2N$$

2.3 First Fit

The *First Fit* algorithm is as follows

- Scan throuh bins $b_1, b_2, \ldots b_i$
- Place item i in the first bin that is large enought to hold it, $c(b_i) \geq s(i)$
- Create a new bin b_{i+1} if there are *no* bins that are large enough

From the lecture slides, we know that for M the number of bins to optimally pack I items, the approximation ratio of First Fit, FF, is

$$SOL_{FF} = \lceil 1.7M \rceil.$$

We start the best fit line at 2^7 samples to eliminate the noise from smaller inputs. We see that the Waste is of order $\mathcal{O}(n^{0.72})$, which is reflective of it's theoretical result. For a large N, In the worst case, we let each bit of

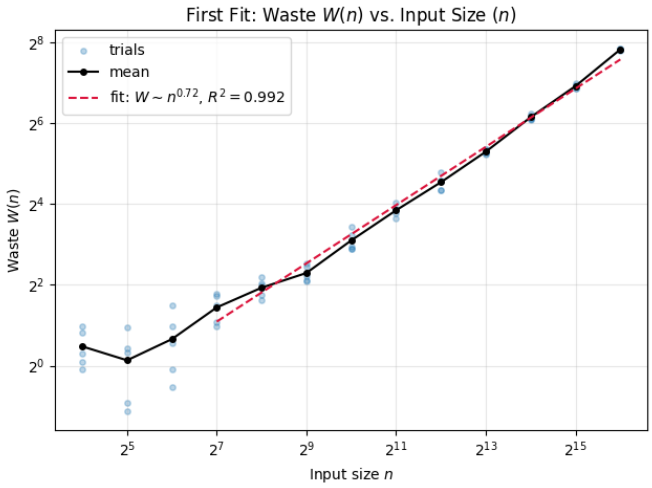


Figure 7: First Fit Waste

waste require a new bucket, and as such we would be bounded by $N_{OPT} + 0.72N_{FF} = 1.72N$, supporting the theoretical result.

2.4 Best Fit

Best Fit works as follows:

- For item i, find the bin b^* with the least remaining capacity that still fits:
$$b^* = \underset{b}{\operatorname{argmin}} \{c(b) - s(i) \mid c(b) \geq s(i)\}$$
where $c(b)$ is the remaining capacity of bin b and $s(i)$ is the size of item i.
- If no such bin exists, open a new one.

The theoretical approximation ratio for *Best Fit*, BF, as suggested by Dosa and Sgall (2014), is
$$SOL_{BF} = k \leq \lfloor 1.70M \rfloor$$

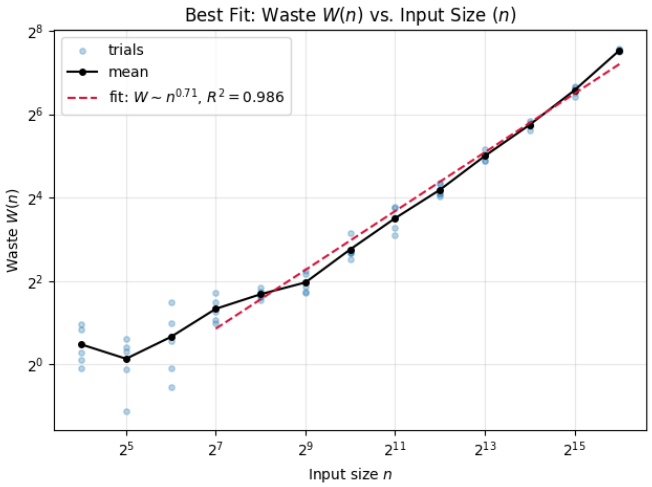


Figure 8: Best Fit Waste

We once again begin fitting the data at 2^7 samples to reduce noise. Notice that there is less variance amoung the trials (less spread out) than in First Fit, but our R^2 value is lower. This suggests that Best Fit may be a more consistent option than first fit. We see that the empirical waste follows $\mathcal{O}(n^{0.71})$, and by the same reasoning as *Next Fit* and *First Fit*, for a large N support
$$1N_{OPT} + 0.71N_{BF} = 1.71N$$

which is a bit above the suggested lower bound, but close enough to call for statistical variance.

2.5 First Fit Decreasing

The *First Fit Decreasing* algorithm is

- Sort the items I in descending order
- Run the *First Fit* algorithm [2.3] on I

Compared to all previous algorithms (*Next Fit*, *First Fit* and *Best Fit*), *First Fit Decreasing* has a wildly different waste-complexity.

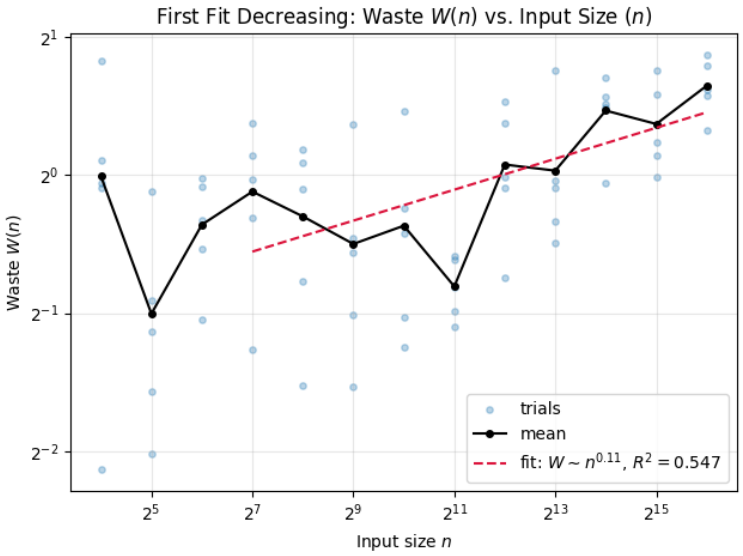


Figure 9: First Fit Decreasing Waste

Note that the scaling of the axis: the trials would appear much closer together if the scaling was the same as the first three algorithms. The empirical waste complexity is $\mathcal{O}(n^{0.11})$, which is much must better than any previous algorithm. Sorting the list in decreasing order lets us more tightly pack (put more items in) the first bins, which has a profound impact on the waste. We should note, however, that *FFD* required us to **sort** the input, which is at best $\mathcal{O}(n \log n)$ using Tim-Sort. While the waste was significantly lower, the runtime might not be.

2.6 Best Fit Decreasing

The *Best Fit Decreasing* algorithm is

- Sort the items I in descending order
- Run the *Best Fit* algorithm [2.4] on I

In excactly the same way as *First Fit Decreasing* *Best Fit Decreasing* has a very very low waste-complexity.

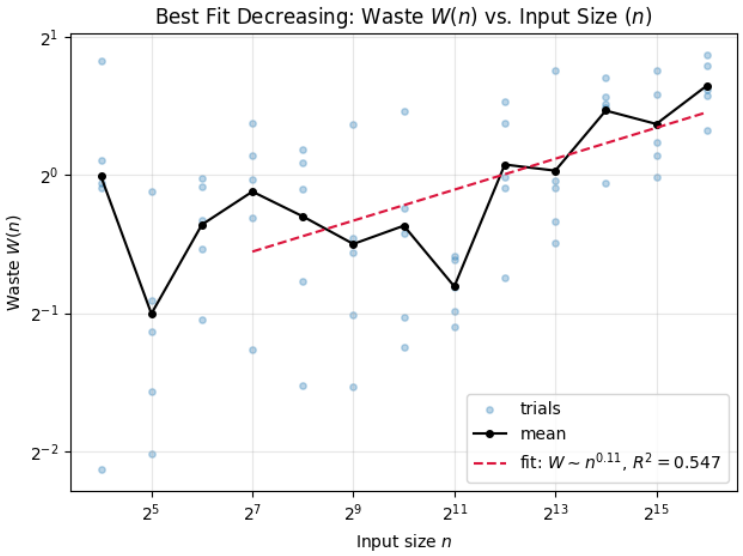


Figure 10: Best Fit Decreasing Waste

Where the empirical waste is order $\mathcal{O}(n^{0.11})$, the same as in *FFD*. Interestly enough, the plots are *exactly* the same. Even considering the individual trials, both graphs are identical. I conjecture that this is no coincidence. That, in fact, each bin may not be filled the exact same way, but the number of bins is always the same, given the input distribution. (expanded upon in [2.7]).

2.7 Conclusion

There are two factors to consider when determining the *best* bin-packing algorithm: *run time* and *waste*.

Let's first rule out *Next Fit* since it's empirical waste complexity was $\mathcal{O}(n^{0.99})$, satisfying its theoretical approximation ratio of $2M$ for M optimal bins. It's simply not a good enough approximation, even though the run-time of the algorithm is the fastest at $\mathcal{O}(n)$.

First fit and *Best fit* perform quite well, with *best fit* slightly edging out the *first fit* with waste complexity $\mathcal{O}_{FF}(n^{0.72})$ *vs.* $\mathcal{O}_{BF}(n^{0.71})$. This makes sense since *best fit* should, in theory, have tighter packings.

The real decision is between *First fit decreasing* and *best fit decreasing*. Both of the graphs (figures 9, 10) are nearly identical, with empirical waste complexity $\mathcal{O}(n^{0.11})$ in both cases. As conjectured in section [2.6], this is likely not a coincidence: once the input is sorted in decreasing order, the largest items dominate the packing structure. Since both algorithms have the same waste complexity, we have to pivot to runtime analysis. *Both* algorithms incur a $\mathcal{O}(n \log n)$ sort cost using `Tim Sort` to sort the items in decreasing order. *BFD* and *FFD* both use a `Zip Zip Tree`, but the augments are different. Asymptotically, the two are equivalent, but *FFD* may have a smaller constant factor since it accepts the *first* feasible bin, rather than the tightest fit one. As such **First Fit Decreasing** is the best bin-packing algorithm.

3 Network Algorithms

This report analyzes structural properties of Barabasi-Albert randomly generated networks. It explores diameter, clustering, and the degree distribution benchmarked on empirical data/results.

3.1 Introduction

Scientists from all fields are interacting and collaborating to understand the importance of, and structure behind, graphs and networks. Graphs enable us to model social interactions, design complex distributed networks, and model many biologial interactions. In short, Networks are *everywhere*.

3.1.1 Structure

We aren't just concerned with graphs as a *construct*, but as a medium to exploit certain structures. These structures, *topologies*, reveal communities, "important" vertices, and many more aspects. This report explores three structures in particular, namely the

- ◇ Degree Distribution
- ◇ Clustering Coefficient
- ◇ and Diameter

of various networks. Each of these structures highlight important variations in meaning, and different graph networks exhibit each structure differently. We use often these structures to characterize certain networks. The canonical example is derived from Travers and Milgram's *An Experimental Study of the Small World Problem*. By instructing participants in Nebraska to send letters to a stockbroker in Boston by *only* sending letters to friends, they showed that the average number of hops for successful letters was between 5 and 6. Formulated as a graph problem, we say the small world network as a small "diameter." As such, the ability to classify graphs based on their structure has many real-world applications, and is the backbone of analysis in many fields.

3.1.2 Barabasi-Albert

Gathering graph's is not as trivial as you may think, and as it turns out, being able to *randomly* generate network structures to simulate real-world

phenomena is of paramount imporance. As such, this report examines one way of generating these graphs: **Barabasi-Albert**.

Algorithm 1 Barabási-Albert (Preferential Attachment)

```
1: procedure PREFERENTIAL ATTACHMENT(n, d)
2:     G = ({0, ..., n − 1}, E)
3:     M ← array of length 2 · n · d
4:     for v ← 1 to n − 1 do
5:         for i ← 0 to d − 1 do
6:             M[2(v · d + i)] ← v
7:             Draw r ∈ {0, ..., 2(v · d + i)} uniformly at random
8:             M[2(v · d + i) + 1] ← M[r]
9:         end for
10:    end for
11:    E ← ∅
12:    for i ← 0 to n · d − 1 do
13:        E ← E ∪ {M[2i], M[2i + 1]}
14:    end for
15:    return G
16: end procedure
```

Pseudocode This algorithm is also called "preferential attachment," since we "prefer" earlier vertices instead of later ones. Formally, the probability p_i that a new node is connected to node i is

$$p_i = \frac{d_i}{\sum_j d_j} \qquad d_k \text{ is the degree of the } k^{th} \text{ node}$$

And the denominator grows as $|V| \to N$, for $N >> n > 0$. We are guaranteed a degeneracy of d, since by induction, if we pop the last vertex added, it's connected to d other vertices. Continuing this down, we see that the graph's degeneracy is determined precisely by d. Preferential attachment produces a *power law* distribtution, where the probability $\mathbb{P}$ over a quantity X is

$$\mathbb{P}\{X\} \propto \frac{1}{X^\beta} \qquad \beta \in \mathbb{R}$$

3.2 Degree Distributions

3.2.1 Definition

The degree distribution $\mathbb{P}(d)$ of a network $G = (V, E)$ is the fraction of nodes with degree d in the network.

$$\mathbb{P}(d) = \frac{|\{v \in G.V \mid \deg(v) = d\}|}{n} = \frac{n_d}{n}$$

The degree distribution is incredibly important when studying real and theoretical networks. Most networks in the real world are *right skewed*, and some are conjectured to follow a power law. In that case, we say the network is *scale free*, since restricting the network to a subset of its nodes is statistically similar to the network in its totality.

3.2.2 Psuedocode

Algorithm 2 Degree Distribution

```
1: procedure DegreeDistribution(G)
2:     H = [0, 0, ..., 0],        |H| = n
3:     for all v ∈ G.V do
4:         d = deg(v)
5:         H[d] += 1
6:     end for
7:     return H
8: end procedure
```

3.2.3 Analysis

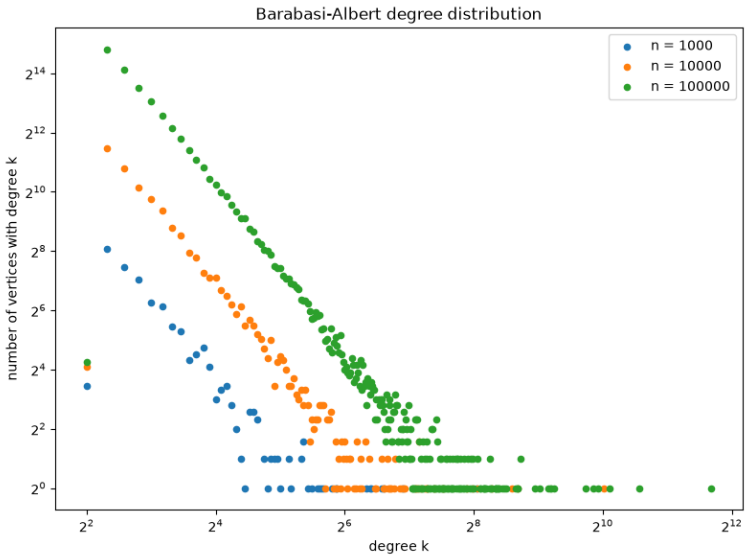


Figure 11: Degree distributions across different network sizes ($n = 1000, 10000, 100000$), plotted on log-log axes.

The degree distribution was analyzed for $n = 1000$, $n = 10000$, and $n = 100000$ vertices, with each distribution overlaid on a single log-log plot. On these axes a power law appears as a straight line, and all three sizes exhibit the same clear linear trend, confirming *scale free* behavior. The **heavy right skew** characteristic of a power law is evident: a large number of low-degree vertices coexist with a handful of very high-degree hubs (the tail extends past $k = 2^{10}$ for $n = 100000$). As n grows the distribution simply extends further along the same slope, so the shape is preserved across scales. As such, the *Barabasi-Albert* random graphs follow a **power law**.

3.3 Clustering Coefficients

3.3.1 Definition

The clustering coefficient is

$$C = \frac{3 \cdot (\text{number of triangles})}{\text{number of 2-edge paths}}.$$

The denominator is cheap; the numerator is computed via a degeneracy ordering so that triangle counting runs in $O(d^2 n) = O(dm)$ expected time rather than the naïve $O(n^3)$.

3.3.2 Psuedocode

Algorithm 3 Number of 2-Edge Paths

```
1: procedure Num2EdgePaths(G)
2:     total ← 0
3:     for all v ∈ G.V do
4:         d ← deg(v)
5:         total ← total + d(d − 1)/2          ▷ paths with v in the middle
6:     end for
7:     return total
8: end procedure
```

Algorithm 4 Degeneracy Ordering

```
1: procedure DegeneracyOrdering(G)
2:     L ← [ ],    H_L ← ∅                ▷ output ordering / vertices placed in L
3:     for all v ∈ G.V do
4:         d_v ← deg(v),    N_v ← [ ]      ▷ N_v: neighbors of v before v in L
5:     end for
6:     for all v ∈ G.V do
7:         D[d_v] ← D[d_v] ∪ {v}          ▷ bucket vertices by current degree
8:     end for
9:     k ← 0
10:    for j ← 1 to n do
11:        i ← min{ i : D[i] ≠ ∅ }
12:        k ← max(k, i)
13:        v ← any vertex in D[i]
14:        D[i] ← D[i] \ {v}
15:        prepend v to L;    H_L ← H_L ∪ {v}
16:        for all w ∈ G.Neighbors(v) with w ∉ H_L do
17:            D[d_w] ← D[d_w] \ {w}
18:            d_w ← d_w − 1
19:            D[d_w] ← D[d_w] ∪ {w}
20:            append w to N_v
21:        end for
22:    end for
23:    return (L, N)
24: end procedure
```

Algorithm 5 Counting Triangles

```
1: procedure CountTriangles(G)
2:     (L, N) ← DegeneracyOrdering(G)
3:     Δ ← 0
4:     for all v ∈ L do
5:         for all unordered pairs {u, w} ⊆ N_v do
6:             if (u, w) ∈ E then                ▷ O(1) adjacency lookup
7:                 Δ ← Δ + 1
8:             end if
9:         end for
10:    end for
11:    return Δ
12: end procedure
```

Algorithm 6 Clustering Coefficient

procedure ClusteringCoefficient(G)
    $\Delta \leftarrow$ CountTriangles(G)
    $P \leftarrow$ Num2EdgePaths(G)
    return $\dfrac{3\Delta}{P}$
end procedure

3.3.3 Analysis

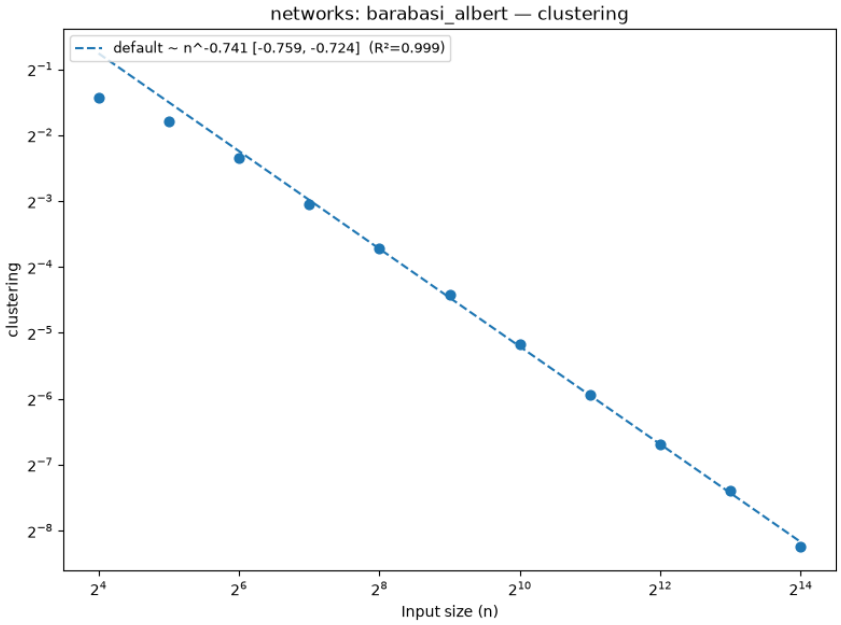


Figure 12: Average clustering vs n (log-log axes) with a fitted power law.

The clustering coefficient, $\mathcal{C}$, measures how "cliquey" the network is. As shown in Figure 12, as n increases, $\mathcal{C}$ decreases. This makes sense, since as we add more vertices via BA [1], they *still* form connections to *only* d other vertices, but the denominator scales with n.

On log-log axes the measurements fall on a near-perfect straight line, so $\mathcal{C}$ follows a **power law** rather than an exponential. The fitted exponent is

$$\mathcal{C} \propto n^{-0.74} \qquad (R^2 = 0.999)$$

The extremely tight fit ($R^2 = 0.999$) confirms that the decay is polynomial in n: each doubling of the graph size multiplies the clustering coefficient by a constant factor of $2^{-0.74} \approx 0.6$, so triangles become steadily rarer as the graph grows but never vanish abruptly.

3.4 Diameters

3.4.1 Definition

The diameter, d, is the greatest distance between any pair of vertices, namely

$$d = \max_{v \in G.V} \max_{u \in G.V} d(v, u)$$

where $d \colon G.V \times G.V \to \mathbb{R}^+ \cup \{0\}$.

3.4.2 Psuedocode

Algorithm 7 Diameter

```
1: procedure HUERISTIC II(G)
2:     r ← 𝒰(1, n)                                  ▷ A random vertex
3:     D_max ← 0
4:     while True do
5:         (w, dist) ← BFS(Graph, r)
6:         if dist > D_max then
7:             D_max ← dist
8:             r ← w
9:         else
10:            break
11:        end if
12:    end while
13:    return D_max
14: end procedure
```

3.4.3 Analysis

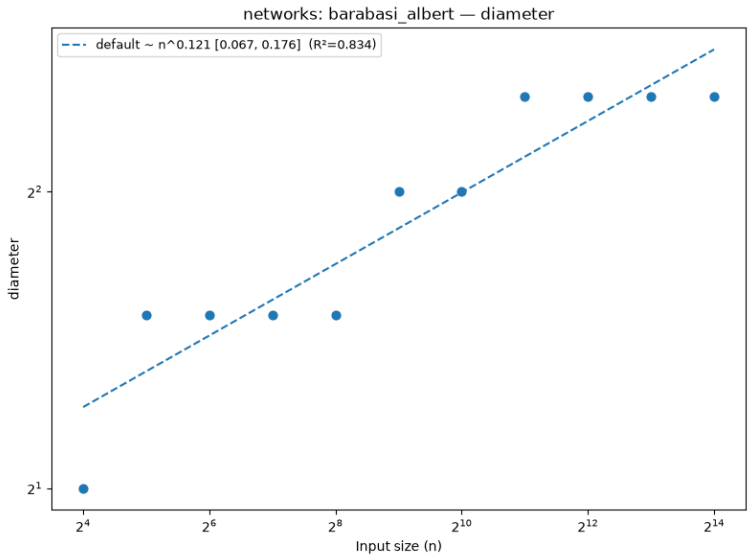


Figure 13: Average Diameter vs n (log-log axes) with a fitted power law.

As shown in Figure 13, the average diameter increases with n, but only very slowly: over roughly three orders of magnitude in n the diameter barely triples. Fitting a power law on log-log axes yields a tiny exponent,

$$d \propto n^{0.12} \qquad (R^2 = 0.834),$$

and such a small exponent with only a moderate fit is exactly the signature of sub-polynomial, essentially *logarithmic*, growth: preferential attachment produces high-degree hubs that act as shortcuts, keeping pairwise distances short even as the graph grows. The jagged, stair-step effect visible at certain values of n is an artifact of the diameter being integer-valued, since the average must accumulate enough samples before incrementing to the next integer. As such, the diameter **is changing as a function of** n, just far more slowly than any power of n.